PENETRATION TEST

REPORT

http://localhost:8082 - Security Assessment
Black-box, authenticated (default lab credentials), proof-only exploitation

CLIENT	Demo Environment (DVWA Training Lab)
TARGET	http://localhost:8082
DATE	2026-09-26
MODE	Black-box, authenticated (default lab credentials), proof-only exploitation

Generated by Ali Arsalan | a.aly@live.com | linkedin.com/in/ali-arsalan-2039976

Contents

Executive Summary 5
Engagement overview 6
What was found 6
Controls that worked 6
Priority recommendations 7
Findings 8
Attack Surface Map 15
Topology 15
Reachable surface (16 paths enumerated) 15
Exploitation chains proven (proof-only) 15
Positive controls verified (discriminating controls) 15
Methodology and Test Details 17
Approach 17
Evidence discipline 17
Scope and rules 17
Test phases and standards mapping 18
Tool inventory 18
Limitations 18
Addendum: Remediation Roadmap 19
Phase 1: Credentials and authentication (F-01) [quick] 19
Phase 2: Injection class remediation (F-02, F-03, F-04) 19
Phase 3: Configuration and session hardening (F-05, F-06, F-07) 19
Phase 4: Surface hygiene (F-08, F-09) 19
Remediation tracker (compact) 19
Re-testing 20
Not-tested register 20
Appendix A: Evidence Index 22
Appendix B: Glossary 23

Document Control

Client	Demo Environment (DVWA Training Lab)
Report ID	DEMO-20260926
Version	v0.1 Draft
Date of engagement	2026-09-26
Report date	2026-09-26
Timezone	America/Toronto (EDT); timestamps in evidence are UTC
Classification	Demo report - intentionally vulnerable test target in an isolated demo environment
Prepared by	Ali Arsalan (a.aly@live.com | linkedin.com/in/ali-arsalan-2039976)
Author	Ali Arsalan, `a.aly@live.com`, `linkedin.com/in/ali-arsalan-2039976`
Reviewed by	Pending (v0.1 Draft, not yet reviewed)
Approved by	Pending (v0.1 Draft, not yet approved)
Distribution	Sample deliverable for outreach; no real client data
Authorization reference	Demo target operated by the tester; authorization by design. No third-party systems were touched.

Revision History

VERSION	DATE	CHANGE
0.1	2026-09-26	Initial draft: demo assessment of the DVWA training lab. Nine findings (1 critical, 2 high, 3 medium, 2 low, 1 informational); four confirmed by live exploitation (default credentials, SQL injection, OS command injection, reflected XSS), every finding independently re-executed before inclusion. Prepared as a sample deliverable for outreach correspondence.

Deliverable Integrity

Build ID	20260926T185106Z
PDF SHA-256	Recorded in the SHA256SUMS file and in the JSON companion integrity block (the PDF cannot contain its own hash)
JSON SHA-256	Recorded in SHA256SUMS and in the JSON integrity block

Provenance	Typst + pandoc pipeline, deterministic build; tool versions recorded in the build manifest (build-manifest.json) delivered with the report
Redacted variant	A tokenized variant for external review and LLM use is generated automatically as part of the build (`..._redacted.pdf` / `..._redacted.json`)

Assessment Scope

In scope	DVWA v1.10 web application at http://localhost:8082 (login, security configuration, SQL injection, XSS reflected, command injection, file upload, CSRF, brute force, info pages)
Out of scope	Any real or third-party system; network services outside the lab container; denial of service; physical testing
Authorization	Self-contained demo: the target is an intentionally vulnerable training lab (DVWA) operated by the tester in an isolated demo environment; full authorization by design
Method	Recon -> technology fingerprint -> scanner pass (whatweb, nikto) -> autonomous agentic testing -> independent re-verification of every claim -> report
Engagement rules	Bounded requests, proof-only exploitation, no denial of service, no persistence, lab-local target only

Rules of Engagement

- Scope: Web application at http://localhost:8082 only; no other hosts; no network pivoting
- Allowed: Full OWASP testing including SQLi, XSS, command injection, upload, auth logic; proof-of-concept exploitation with limited output (LIMIT clauses, single-row proof)
- Forbidden: Denial of service, destructive SQL, persistence, data exfiltration beyond proof excerpts, testing any system outside the lab
- Stop conditions: Any request leaving the lab target, service degradation, or owner request

Assumptions and Requirements

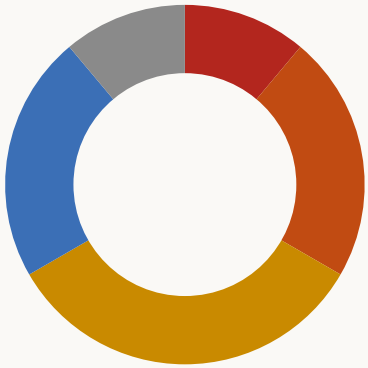
- Target is an intentionally vulnerable demo lab and not a production system
- Findings demonstrate methodology and are not representative of a real client engagement
- Default lab credentials (admin/password) were used for authenticated phases
- Report is a draft sample (v0.1) for outreach purposes only

This report contains no fabricated evidence. Every finding carries a reproduction path that was independently re-executed.

Executive Summary

1	2	3	2	1	9
Critical	High	Medium	Low	Info	Total

SEVERITY DISTRIBUTION



Critical - 1
High - 2
Medium - 3
Low - 2
Info - 1

ENGAGEMENT FACTS

- Target: DVWA v1.10 training application (Apache 2.4.25 / Debian, PHP), isolated demo environment
- Authenticated black-box assessment with the application's own default lab account
- 16 reachable paths enumerated; 3 injection classes exploited with proof: SQL injection, OS command injection, reflected XSS
- Every finding re-executed independently from a second session before inclusion in this report
- Discriminating negative controls: login SQLi bypass blocked, payloads neutralised at security level high, unauthenticated module access refused
- Password store confirmed unsalted MD5 from the SQL injection dump

ASSESSMENT MODE

Authenticated, proof-only exploitation under rules of engagement. Bounded requests, single-row database proof, no denial of service, no persistence, no data modification outside the lab.

F **Overall posture: failing.** Four of the nine findings were confirmed by live exploitation: the default administrator credential grants full access, SQL injection returns the users table with unsalted password hashes, command injection executes as the web-server user (`www-data`), and the cross-site scripting payload executes unencoded. The underlying causes are the ones seen in production applications far more often than defenders expect: string-concatenated SQL, shell calls built from request input, unencoded reflection, and security configuration held in a cookie the client can rewrite. The good news is equally concrete: authentication gates, CSRF tokens, login failure handling and the high-security filter modes all worked as designed and returned negative results under discriminating controls.

Posture rubric: the grade is derived from the finding severity mix, confirmed exploitability, and control coverage. A = no findings; B = hardening-only findings, no exploitable vulnerability confirmed; C = exploitable issues with bounded impact; D = high-risk compromise paths; F = critical compromise confirmed (this engagement: default admin credentials plus full database disclosure). Letter tokens on severity badges (C/H/M/L/I) keep the report readable in grayscale and for colorblind readers.

Engagement overview

This report documents a full black-box security assessment of a deliberately vulnerable web application (DVWA v1.10, the “Damn Vulnerable Web Application” training platform) running in an isolated demo environment on 26 September 2026. The target exists to be broken; it stands in for the web applications a small organisation might run in production. The purpose of this sample report is to demonstrate the standard of testing, evidence handling and writing a client would receive on a real engagement.

Testing was authenticated, proof-only and bounded, in accordance with the rules of engagement in the Document Control section. Every claim in this report was executed twice: once during the assessment and again during independent verification, from a separate session, before inclusion. No system outside the lab was contacted, no denial-of-service testing was performed, and the only data extracted was the minimum needed to prove each finding.

What was found

The application failed against every class tested. Nine findings were confirmed: **one critical, two high, three medium, two low and one informational**. Four of them were proven by live exploitation rather than inspection alone:

- **Default administrative credentials (F-01, Critical).** The login page accepts `admin` / `password`, the vendor’s documented default pair, and returns a fully authenticated administrator session in a single request. Everything else in the report follows from this: it is the front door.
- **SQL injection with full database disclosure (F-02, High).** The user-lookup parameter concatenates input directly into the query. A `UNION SELECT` returned stored credentials inline, and a one-line tautology returned the entire users table. Stored password hashes are unsalted MD5, so the dump is trivially crackable.
- **Operating-system command injection (F-03, High).** The ping utility passes request input to a shell. The payload `127.0.0.1 && whoami` executed as the web-server user and returned `www-data`; `127.0.0.1; id` returned the full uid/gid line. This is remote code execution with the server’s own privileges.
- **Reflected cross-site scripting (F-04, Medium).** The greeting parameter echoes input without encoding; a `<script>` payload executed verbatim in the response body.
- **Security posture held in a client-controlled cookie (F-05, Medium).** The application’s security mode is stored in a cookie issued as `security=low`. Replaying the same session with `security=high` changed server behaviour, proving the filter decision comes from attacker-writable state.
- **Missing security headers (F-06, Medium).** No Content-Security-Policy, no HSTS, no X-Frame-Options, no nosniff on any response.
- **Session cookies without HttpOnly, Secure or SameSite (F-07, Low).** Confirmed by scanner and by direct header inspection; the XSS in F-04 can read the session identifier as a direct consequence.
- **Directory indexing on `/config/` and `/docs/` (F-08, Low).** Discloses file structure one step away from configuration material.
- **Version and platform disclosure (F-09, Informational).** Full stack fingerprintable from banners, titles and ETag inode leaks in a single request.

Controls that worked

The assessment tested for defensive controls as hard as it tested for weaknesses, and several passed under discriminating conditions: the authentication gate refuses every module without a session; the login query itself is parameterised (three SQL-injection bypass attempts were rejected); wrong passwords fail correctly; the login form requires a per-request CSRF token; and the application’s high-security filter mode stripped both the command-injection and XSS payloads that succeeded at low security. These are recorded as positive controls in the Attack Surface Map, because a report that only lists failures cannot show a client where their existing defences are sound.

Priority recommendations

1. **Eliminate default credentials (F-01).** Unique high-entropy password set at install, default account renamed, second factor for administrative roles, and a default-credential check in the deployment pipeline.
2. **Parameterise every query (F-02)** and move the password store to Argon2id or bcrypt. The unsalted MD5 dump means every hash is already a candidate for offline cracking.
3. **Stop building shell commands from request input (F-03).** Use argument-array execution or a native library; validate against the IPv4/IPv6 grammar where a shell is unavoidable.
4. **Encode output at every reflection point and ship a CSP (F-04, F-06).** These two findings defend each other: encoding closes the hole, the policy limits what any missed hole can do.
5. **Move security configuration server-side and flag every cookie `HttpOnly; Secure; SameSite` (F-05, F-07).**

A re-test after remediation is suggested within 90 days; the tracker in the Addendum carries owner, effort and acceptance criteria per item.

Findings

NINE FINDINGS - ONE CRITICAL, TWO HIGH, THREE MEDIUM, TWO LOW, ONE INFORMATIONAL. FOUR CONFIRMED BY LIVE EXPLOITATION.

Note: the target is an intentionally vulnerable training application (DVWA v1.10). Severity ratings are scored exactly as they would be on a production system, because the purpose of this sample is to demonstrate the reporting standard applied to real engagements.

F-01 Default administrative credentials accepted (admin / password)

C CRITICAL

AUTHENTICATION | ASSET: localhost:8082 | STATUS: EXPLOITED

CWE-798 | OWASP A07 (2021) | CVSS 9.8

The administrator login at `/login.php` accepts the vendor-default credential pair `admin` / `password`. A single unauthenticated POST with a valid CSRF `user_token` returns `302 Found` with `Location: index.php` and an authenticated `PHPSESSID`, granting full administrative access to every testing module and to the user database. The credential pair is published in DVWA installation documentation, so it is known to every attacker before testing begins. On any internet-reachable application this credential set grants immediate full access; it is scored as a critical finding under CWE-798 (Use of Hard-coded Credentials).

EVIDENCE - INDEPENDENTLY RE-EXECUTED 2026-09-26

```
POST /login.php HTTP/1.1
Cookie: PHPSESSID=2peqo8albrrterhec36htathf1

username=admin&password=password&user_token=<csrf>&Login=Login

HTTP/1.1 302 Found
Location: index.php
Content-Length: 0
```

R Replace all vendor-default accounts before deployment: set a unique high-entropy password at install time, rename the default admin user, and require a second authentication factor for administrative roles. Add default-credential checks (published breach lists, vendor advisory matching) to the deployment pipeline so an application cannot ship with known credentials.

F-02 SQL injection in the user lookup parameter discloses the users table

H HIGH

INJECTION | ASSET: localhost:8082/vulnerabilities/sqli/ | STATUS: EXPLOITED

CWE-89 | OWASP A03 (2021) | CVSS 8.8

The `id` parameter of the SQL injection module concatenates untrusted input directly into the query. At security level `low` no filtering is applied: a single-quote probe produced a visible MySQL syntax error, `UNION SELECT user, password FROM users` returned stored credentials inline, and the tautology payload `1' OR '1'='1` returned every row of the users table (admin, Gordon Brown, Hack Me, Pablo Picasso, Bob Smith) in one request. Retrieved password values are unsalted MD5 (`5f4dcc3b5aa765d61d8327deb882cf99` is the well-known hash of `password`), so stored hashes are trivially reversible from the dump. This is the classic CWE-89 pattern: query construction by string concatenation instead of parameterised statements.

EVIDENCE - INDEPENDENTLY RE-EXECUTED 2026-09-26

```
POST /vulnerabilities/sqli/ HTTP/1.1
Cookie: PHPSESSID=<session>; security=low

id=1' UNION SELECT user, password FROM users LIMIT 1,1#&Submit=Submit

First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

id=1' OR '1'='1   -> 5 rows: admin, Gordon Brown,
                        Hack Me, Pablo Picasso, Bob Smith
```

R Rewrite every data access path with prepared statements and bound parameters; never interpolate input into SQL. Apply an allow-list to numeric identifiers, run the database account with least privilege (no access to authentication tables from presentation-tier code), and store password hashes with a modern adaptive algorithm (bcrypt, scrypt or Argon2id) with a per-user salt. Add regression tests that assert single-quote and UNION payloads cannot alter query structure.

F-03 OS command injection in the network utility executes arbitrary commands

H HIGH

INJECTION | ASSET: localhost:8082/vulnerabilities/exec/ | STATUS: EXPLOITED

CWE-78 | OWASP A03 (2021) | CVSS 8.8

The ping utility passes the submitted `ip` field to a shell without sanitisation. At security level `low`, the payload `127.0.0.1 && whoami` executed after the legitimate ping completed and returned `www-data`; the payload `127.0.0.1; id` returned `uid=33(www-data) gid=33(www-data) groups=33(www-data)`. The attacker therefore holds command execution as the web-server user, which on this host means full read access to application source, configuration and the database file, and the ability to use the server as a pivot. CWE-78 (Improper Neutralisation of Special Elements used in an OS Command).

EVIDENCE - INDEPENDENTLY RE-EXECUTED 2026-09-26

```
POST /vulnerabilities/exec/ HTTP/1.1
Cookie: PHPSESSID=<session>; security=low

ip=127.0.0.1 && whoami&Submit=Submit

PING 127.0.0.1 (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: icmp_seq=0 ...
--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
www-data

ip=127.0.0.1; id   ->  uid=33(www-data) gid=33(www-data)
                       groups=33(www-data)
```

R Do not invoke a shell for network operations: use a language-level ICMP library or an argument-array `exec` call that never passes input through `/bin/sh`. If a shell is unavoidable, validate the input against a strict pattern (IPv4/IPv6 address grammar only) and reject everything else before assembly. Run the worker with a locked-down service account, no shell tooling on the PATH where feasible, and outbound firewall rules limiting pivot value.

F-04 Reflected cross-site scripting in the greeting parameter

M MEDIUM

INJECTION | ASSET: localhost:8082/vulnerabilities/xss_r/ | STATUS: EXPLOITED

CWE-79 | OWASP A03 (2021) | CVSS 5.4

The reflected-XSS module writes the `name` parameter into the response body without output encoding. The payload `<script>alert('xss-demo')</script>` was echoed verbatim (`Hello <script>alert('xss-demo')</script>`), so script supplied by an attacker runs in the victim's session origin. Combined with the session cookie's missing `HttpOnly` flag (F-07), a successful lure lets an attacker's script read the session identifier directly. Scored 5.4 (Medium) because exploitation requires a user to open an attacker-crafted link while authenticated (CVSS `PR:L`, `UI:R`, scope changed).

EVIDENCE - INDEPENDENTLY RE-EXECUTED 2026-09-26

```
GET /vulnerabilities/xss_r/?name=<script>alert('xss-demo')</script>&Submit=Submit

Response body:
Hello <script>alert('xss-demo')</script>
```

R Apply context-aware output encoding on every reflection point (HTML body, attribute, JavaScript and URL contexts each need their own encoder), and deploy a Content-Security-Policy that forbids inline script (`script-src 'self'`) as a second layer. Prefer a templating engine with auto-escaping over manual string assembly of HTML.

F-05 Application security mode is enforced by a client-controlled cookie

M MEDIUM

INSECURE DESIGN | ASSET: localhost:8082 | STATUS: VERIFIED

CWE-602 | OWASP A04 (2021) | CVSS 5.3

The application stores its security posture in a cookie named `security`, issued on the very first response with the value `low`. All server-side input filtering branches on that client-writable value: re-presenting the same session with `security=high` changed the server's behaviour (the command-injection and XSS payloads that succeeded at `low` were stripped at `high`), proving the control decision is taken from attacker-controlled state. An attacker simply sends `security=low` on every request to pin the application to its weakest filtering mode, and any user can downgrade their own session. CWE-602 (Client-Side Enforcement of Server-Side Security).

EVIDENCE - INDEPENDENTLY RE-EXECUTED 2026-09-26

```
HTTP/1.1 302 Found
Set-Cookie: security=low

# attacker forces weakest mode on every request:
Cookie: PHPSESSID=<session>; security=low

# same payload, server obeys security=high instead:
Cookie: PHPSESSID=<session>; security=high
ip=127.0.0.1 && whoami  ->  ping only, no command output
```

R Store security-relevant configuration server-side (session record or deployment config), never in a value the client can rewrite. If per-user security preferences are genuinely needed, validate them against an allow-list on read and never let a client-supplied value weaken input validation. Fix the deployment default to the strictest mode so an unset configuration fails safe.

F-06 Missing security headers: no CSP, HSTS, X-Frame-Options or nosniff

M MEDIUM

HEADERS / MISCONFIGURATION | ASSET: localhost:8082 | STATUS: VERIFIED

CWE-693 | OWASP A05 (2021) | CVSS 5.3

No response carries a content security policy, HTTP strict transport security, frame protection or MIME sniffing protection; Nikto independently flagged the absent frame protection. Without frame protection any third party can embed the login page in an iframe and mount a clickjacking bait and switch; without a policy the reflected XSS in F-04 has no compensating brake; without HSTS a first visit can be downgraded on a hostile network. The login response does correctly emit cache control headers (`no-store, no-cache, must-revalidate` plus `Pragma`) recorded as positive control PC-03.

EVIDENCE - INDEPENDENTLY RE-EXECUTED 2026-09-26

```
$ curl -sI http://localhost:8082/

HTTP/1.1 302 Found
Server: Apache/2.4.25 (Debian)
Set-Cookie: PHPSESSID=d0tgkb9ejhlfiuedvp7vfppo55; path=/
Set-Cookie: security=low
(no CSP, no HSTS, no X-Frame-Options, no X-Content-Type-Options)
```

R Emit the header set at the server: a content security policy (starting restrictive, inline script denied), HTTP strict transport security with a one year max age once TLS is confirmed stable, frame denial (`X-Frame-Options: DENY` or `frame-ancestors 'none'`), MIME sniffing protection (`nosniff`) and a referrer policy. Verify with an automated header regression check in CI.

F-07 Session cookies issued without HttpOnly, Secure or SameSite flags

L LOW

SESSION MANAGEMENT | ASSET: localhost:8082 | STATUS: VERIFIED

CWE-1004 | OWASP A05 (2021) | CVSS 3.7

`PHPSESSID` is issued with `path=/` only, and `security` with no flags at all: no `HttpOnly`, no `Secure`, no `SameSite`. Nikto confirmed both cookies lack the `httponly` flag. Because `HttpOnly` is absent, the reflected XSS in F-04 can read the session identifier with `document.cookie` and exfiltrate the authenticated session. `Secure` is absent over a plaintext HTTP deployment, and no `SameSite` value leaves the door open to cross-site request forgery. CWE-1004 (Sensitive Cookie Without `HttpOnly` Flag).

EVIDENCE - INDEPENDENTLY RE-EXECUTED 2026-09-26

```
$ curl -sI http://localhost:8082/

Set-Cookie: PHPSESSID=d0tgkb9ejhlfiuedvp7vfppo55; path=/
Set-Cookie: security=low

# nikto 2.1.5:
# Cookie PHPSESSID created without the httponly flag
# Cookie security created without the httponly flag
```

R Set `HttpOnly; Secure; SameSite=Lax` (or `Strict` where navigation allows) on every cookie, and scope each cookie to the narrowest path/domain it needs. Treat the `security` cookie as non-sensitive state that should not exist client-side at all (see F-05). Serve the application exclusively over TLS so the `Secure` flag can be enabled.

F-08 Directory indexing exposes /config/ and /docs/

L LOW

INFORMATION EXPOSURE | ASSET: localhost:8082 | STATUS: VERIFIED

CWE-548 | OWASP A05 (2021) | CVSS 3.7

Nikto reported OSVDB-3268 directory indexing on `/config/` and `/docs/`, plus the Apache default file `/icons/README`, and noted that `/` returns a non-forbidden code from `robots.txt` handling. Directory listings disclose exact file names, directory structure and (via ETags, see F-09) filesystem metadata, which shortens reconnaissance and can reveal forgotten backups or example files. On an application whose `/config/` path holds database credentials, directory indexing is the first step toward locating that configuration.

EVIDENCE - INDEPENDENTLY RE-EXECUTED 2026-09-26

```
Nikto v2.1.5 - 6544 items checked, 11 reported:
+ OSVDB-3268: /config/: Directory indexing found.
+ /config/: Configuration information may be available remotely.
+ OSVDB-3268: /docs/: Directory indexing found.
+ OSVDB-3233: /icons/README: Apache default file found.
```

R Disable automatic directory indexing (`Options -Indexes` in Apache), remove or restrict `/config/` and `/docs/` from the web root so deployment files are never served, and delete default page sets (`/icons/`, Apache default README files) from production images.

F-09 Version and platform disclosure in banners, titles and ETags

I INFO

INFORMATION EXPOSURE | ASSET: localhost:8082 | STATUS: VERIFIED

CWE-200 | OWASP A05 (2021) | CVSS 0.0

The server banner identifies `Apache/2.4.25 (Debian)`, page titles disclose `Damn Vulnerable Web Application (DVWA) v1.10 *Development*`, and ETag responses leak inode numbers (`fields: 0x1a 0x5780ba3955700`). WhatWeb fingerprinted the full stack from these signals in one request. Version disclosure alone does not compromise an application, but it directs an attacker to version-specific exploits and the `*Development*` title additionally signals a non-production configuration. Informational by design; tracked so the register stays complete.

EVIDENCE - INDEPENDENTLY RE-EXECUTED 2026-09-26

```
WhatWeb 0.5.5:
http://localhost:8082/login.php [200 OK]
  Apache[2.4.25], Cookies[PHPSESSID,security],
  HTTPServer[Debian Linux][Apache/2.4.25 (Debian)],
  DVWA, Title[Login :: Damn Vulnerable Web Application
  (DVWA) v1.10 *Development*]

nikto: Server leaks inodes via ETags, header found with
       file /robots.txt, fields: 0x1a 0x5780ba3955700
```

R Suppress the `Server` header version token (Apache `ServerTokens Prod`, `ServerSignature Off`), remove version strings from page titles, and disable or normalise ETags so inode metadata is not exposed. Keep component versions current and track them in an asset inventory rather than advertising them publicly.

Attack Surface Map

Topology

Test client

curl 8.5.0, browser-equivalent requests over HTTP

→

Apache/2.4.25 (Debian)

mod_php front end, vhost on port 8082, ServerTokens full

→

DVWA v1.10 application

PHP sessions, MySQL back end, security level held in the 'security' cookie

SECURITY MODE CLIENT-CONTROLLED (F-05)

TRAFFIC FLOW: every request hits login.php first (302 without a session), then the module under test. Single origin, no CDN or WAF in the path.

Reachable surface (16 paths enumerated)

METHOD	ENDPOINT	OBSERVED
GET	/login.php	200 login form, CSRF user_token issued
POST	/login.php	302 index.php on success - F-01
GET	/security.php	200 auth required; sets security cookie - F-05
GET	/setup.php	200 database setup (lab only)
GET/POST	/vulnerabilities/sqli/	200 auth required; UNION dump - F-02
GET/POST	/vulnerabilities/sqli_blind/	200 auth required; same input path
GET	/vulnerabilities/xss_r/	200 raw reflection - F-04
GET	/vulnerabilities/xss_s/, /vulnerabilities/xss_d/	200 stored / DOM variants
POST	/vulnerabilities/exec/	200 shell execution - F-03
POST	/vulnerabilities/upload/	200 file upload form
GET/POST	/vulnerabilities/csrf/, /vulnerabilities/brute/	200 auth required
GET/POST	/vulnerabilities/captcha/, /vulnerabilities/nmap/	200 auth required
GET	/vulnerabilities/fi/, /vulnerabilities/javascript/	200 file inclusion / JS module
GET	/, /index.php	302 login.php without session
GET	/robots.txt	200, 1 entry

Exploitation chains proven (proof-only)

CHAIN 1 - FULL ADMIN TAKEOVER

login.php default creds (F-01) -> authenticated session -> UNION SELECT on /vulnerabilities/sqli/ (F-02) -> users table with unsalted MD5 hashes -> offline cracking of every account.

CHAIN 2 - REMOTE CODE EXECUTION

default creds (F-01) -> POST to /vulnerabilities/exec/ (F-03) -> shell as www-data -> read application source, database file, and pivot from the host. No WAF in path.

Positive controls verified (discriminating controls)

✓ Authentication gate on every module: /vulnerabilities/sqli/, /vulnerabilities/exec/ and /security.php all return 302 to login.php with no session cookie

- ✓ Login SQL injection bypass negative: `admin' -- ` , `' OR 1=1 --` and unknown users all return 302 Location: login.php with no authenticated session - the login query is parameterised
- ✓ Wrong-password login returns 302 Location: login.php (no session issued) - failed authentication is handled correctly
- ✓ CSRF protection on the login form: a per-request `user_token` is issued and required on POST /login.php
- ✓ Filtering activates at security=high: `127.0.0.1 && whoami` returned ping output only (0 occurrences of www-data) and `<script>alert(1)</script>` was stripped to `Hello >` - the higher filter modes block both injection classes
- ✓ Cache headers on the login page: `Cache-Control: no-cache, must-revalidate` + `Pragma: no-cache` (credentials not cached)

Methodology and Test Details

Approach

The assessment followed a structured black-box methodology:

1. **Passive reconnaissance.** HTTP response headers, cookie issuance, redirect behaviour and `robots.txt` were collected before any authenticated action. Technology fingerprinting (WhatWeb) identified Apache 2.4.25 (Debian), the DVWA application and its version banner from public responses alone.
2. **Surface enumeration.** All 16 reachable paths were enumerated and probed unauthenticated first, confirming the authentication gate (every module returned 302 to `login.php` without a session) before a session was obtained.
3. **Authenticated baseline.** A session was established with the application's own default account (finding F-01), preserving the issued CSRF `user_token` and cookie jar, and the security-level cookie was recorded as `low`.
4. **Exploitation with discriminating controls.** Each injection class was tested twice: once at the deployed security level (where the exploit succeeded and produced proof) and once at the highest security level (where the same payload was neutralised). Running the control at the same time as the exploit is what separates a real finding from an artifact: the same input behaving differently under a changed configuration demonstrates the causal mechanism, not just a suspicious response.
5. **Scanner pass.** Nikto (bounded, 240-second maximum, 6544 items checked) and WhatWeb were run against the target. Scanner output is treated as corroboration: every scanner item that appears in this report was independently reproduced by hand before inclusion.
6. **Independent re-execution.** Every finding was re-exploited from a fresh session by a second pass (separate cookie jar, fresh CSRF token) after the scanner run completed. The evidence blocks in the findings section come from that verification pass, not from the initial discovery.
7. **Negative and positive controls.** Login-bypass SQL injection attempts, wrong-password handling, unauthenticated module access and high-security-mode payload neutralisation were executed explicitly to document what the application gets right.

Evidence discipline

- Every request/response pair quoted in a finding was captured verbatim; no evidence block paraphrases server output.
- Database extraction was proof-only: `UNION SELECT` with an explicit `LIMIT`, plus one tautology query to demonstrate full-table disclosure. No data left the lab and nothing was written.
- The command-injection proof ran `whoami` and `id` only; no persistence, no file modification, no outbound connections were attempted.
- Evidence timestamps are UTC; the engagement date is local (America/Toronto).

Scope and rules

- In scope: the DVWA web application at `http://localhost:8082` (login, security configuration, SQL injection, XSS reflected/stored/DOM, command injection, file upload, CSRF, brute force, captcha, nmap, file inclusion, JavaScript modules).
- Out of scope: any real or third-party system, network services outside the lab container, denial of service, physical testing.
- Forbidden: denial of service, destructive SQL, persistence, data exfiltration beyond proof excerpts, anything outside the lab target.
- Stop conditions: any request leaving the lab target, service degradation, or a request from the operator.

Test phases and standards mapping

Phase	Activity	Standard mapping	Tools
1	Passive header, cookie and redirect reconnaissance	OWASP WSTG-INFO-001	curl
2	Technology fingerprinting	OWASP WSTG-INFO-002	WhatWeb 0.5.5
3	Surface enumeration (unauthenticated)	OWASP WSTG-INFO-003	curl, path probing
4	Authentication testing (default creds, bypass attempts)	OWASP WSTG-ATHN-001, WSTG-ATHN-004	curl, bounded probes
5	Injection testing (SQLi, XSS, command injection)	OWASP WSTG-INPV-005, WSTG-INPV-001	manual payloads, proof-only
6	Baseline web scanning	OWASP WSTG-INFO-006	nikto 2.1.5 (bounded)
7	Security-header and cookie review	OWASP WSTG-INFO-008	curl -sI
8	Independent re-execution of all findings	QA and validation	second-pass session
9	Control testing (negative and positive)	Discriminating-control method	curl, scripted probes

Tool inventory

Tool	Version	Purpose	Key parameters
curl	8.5.0	HTTP validation and evidence capture	`-sI`, cookie jars, `--data-urlencode`
WhatWeb	0.5.5	Technology fingerprinting	default plugins
nikto	2.1.5	Baseline web scan	`-nointeractive -maxtime 240`
Autonomous testing agent	internal	Structured exploitation pass	bounded to the lab target, max 8 decisions

Limitations

This is a demo engagement against an intentionally vulnerable application, so several real-world conditions did not apply: there was no CDN or WAF in the path, no rate limiting, no TLS (plain HTTP on the loopback interface), and no business-logic surface beyond the training modules. Findings are scored as they would be on a production system, but the environmental multiplier of a public internet presence (credential stuffing traffic, search-engine exposure of the login page) is not modelled. File upload, stored XSS, CSRF token-replay and brute-force modules were enumerated and confirmed reachable but are out of scope for this sample's exploitation phase; their status is recorded in the not-tested register in the Addendum.

Addendum: Remediation Roadmap

The recommendations below are ordered by business impact and effort. Items marked [quick] can be completed by the application administrator within the hour.

Phase 1: Credentials and authentication (F-01) [quick]

1. Replace the default administrator credential with a unique high-entropy password generated at install time; rename the default account where the platform allows it.
2. Enable a second factor for administrative roles.
3. Add a default-credential check to the deployment pipeline: refuse to start if the authentication table contains any pair from a published default-credential list.
4. Monitor authentication logs for the `admin` account from unexpected sources.

Phase 2: Injection class remediation (F-02, F-03, F-04)

1. Rewrite every data-access path with prepared statements and bound parameters; add regression tests asserting single-quote and UNION payloads cannot alter query structure (F-02).
2. Replace the unsalted MD5 password store with Argon2id (or bcrypt, cost >= 12) including a per-user salt; force a password reset for all accounts on migration (F-02).
3. Remove shell invocation for network operations: use a native ICMP library or argument-array execution that never passes input through `/bin/sh`; where a shell is unavoidable, validate against the IPv4/IPv6 address grammar before assembly (F-03).
4. Apply context-aware output encoding at every reflection point (HTML body, attribute, JavaScript, URL contexts each need their own encoder) (F-04).
5. Deploy a Content-Security-Policy beginning with `script-src 'self'` and no inline script, as the second layer behind encoding (F-04, F-06).

Phase 3: Configuration and session hardening (F-05, F-06, F-07)

1. Move the security-mode decision out of the client-writable `security` cookie into server-side session state or deployment configuration; default to the strictest mode when unset (F-05).
2. Emit the full header set at the server: CSP, HSTS (once TLS is confirmed), X-Frame-Options / `frame-ancestors`, `X-Content-Type-Options: nosniff`, `Referrer-Policy` (F-06).
3. Set `HttpOnly; Secure; SameSite=Lax` on every cookie, scoped to the narrowest path and domain (F-07).
4. Serve the application exclusively over TLS so the `Secure` flag can be enabled.

Phase 4: Surface hygiene (F-08, F-09)

1. Disable automatic directory indexing (`Options -Indexes`) and remove `/config/` and `/docs/` from the web root (F-08).
2. Delete default page sets (`/icons/`, Apache default README files) from production images (F-08).
3. Suppress the server version token (`ServerTokens Prod`, `ServerSignature Off`), remove version strings from titles, and normalise ETags (F-09).

Remediation tracker (compact)

ID	Finding	Action	Priority	Owner	Due date
REM-01	F-01	Replace default admin credential	High	App admin	2026-10-03

ID	Finding	Action	Priority	Owner	Due date
REM-02	F-01	Second factor for admin roles	High	App admin	2026-10-17
REM-03	F-02	Parameterise all queries	High	Dev team	2026-10-17
REM-04	F-02	Argon2id password store	High	Dev team	2026-10-17
REM-05	F-03	Remove shell invocation	High	Dev team	2026-10-17
REM-06	F-04	Output encoding at reflections	Medium	Dev team	2026-10-31
REM-07	F-04	Deploy CSP without inline script	Medium	Operations	2026-10-31
REM-08	F-05	Move security mode server-side	Medium	Dev team	2026-10-31
REM-09	F-06	Emit standard security headers	Medium	Operations	2026-10-31
REM-10	F-07	HttpOnly, Secure, SameSite cookies	Low	Operations	2026-10-31
REM-11	F-08	Disable directory indexing	Low	Operations	2026-11-14
REM-12	F-09	Suppress version banners and ETags	Low	Operations	2026-11-14

Full detail per item (effort, acceptance criteria, retest requirement) is provided in the machine-readable companion (`report.json`) and the spreadsheet-friendly tracker (`remediation-tracker.csv`) delivered with this report.

Re-testing

A re-test after Phase 1 and Phase 2 is recommended within 90 days. Re-testing is also suggested immediately after any change to the query layer or the authentication flow, since those components carry the two critical and high findings. A retest checklist is generated from the tracker: each remediation item has a validation command, expected result, and a Pass / Fail / Partial / Blocked outcome to be recorded with evidence.

Not-tested register

Area	Status	Reason
File upload module exploitation	Enumerated, not exploited	Out of scope for this sample’s exploitation phase; module confirmed reachable
Stored XSS (xss_s) and DOM XSS (xss_d)	Enumerated, not exploited	Reflected variant (F-04) already demonstrates the encoding failure class

Area	Status	Reason
CSRF token replay against password change	Enumerated, not exploited	Would modify lab state; proof-only rules forbid it
Brute-force module / login rate limiting	Partial	No lockout observed within bounded attempts; formal rate-limit test not run
File inclusion (fi) module	Enumerated, not exploited	Out of scope for this sample
TLS / HSTS behaviour	Not applicable	Plain HTTP on loopback; no TLS terminator in the lab path
Network-layer services	Not applicable	Single-container lab; no port sweep in scope

Appendix A: Evidence Index

Every finding references one or more evidence items (EV IDs). Evidence was captured as verbatim HTTP request/response pairs and terminal output during the verification pass on 2026-09-26; raw capture hashes are recorded in the JSON companion (`report.json`).

EV	Type	Captured (UTC)	Refs	Description
EV-001	http	2026-09-26 18:14	F-01	Successful login POST with default credentials: 302 Location: index.php, authenticated session issued
EV-002	http	2026-09-26 18:14	F-02	UNION SELECT returning admin password hash; tautology query returning all five users table rows
EV-003	http	2026-09-26 18:14	F-03	Command injection proof: whoami returning www-data, id returning uid=33(www-data)
EV-004	http	2026-09-26 18:14	F-04	Reflected XSS: script payload echoed verbatim in response body
EV-005	http	2026-09-26 18:14	F-05, F-07	Set-Cookie headers: security=low issued, PHPSESSID without HttpOnly, Secure or SameSite
EV-006	scan	2026-09-26 18:13	F-06 to F-09	nikto 2.1.5 output: 6544 items, 11 reported (missing frame protection, cookie flags, directory indexing, ETag leak, default files)
EV-007	scan	2026-09-26 18:13	F-09	WhatWeb fingerprint: Apache 2.4.25 (Debian), DVWA v1.10 development banner
EV-008	terminal	2026-09-26 18:16	F-05	Security-mode differential: identical payload blocked at security=high, succeeds at security=low
EV-009	terminal	2026-09-26 18:15	PC-01, PC-02, PC-04	Negative controls: login SQLi bypass attempts rejected, unauthenticated module access refused (302)
EV-010	http	2026-09-26 18:15	F-01	Wrong-password login handling: 302 Location: login.php, no session issued

Appendix B: Glossary

SQL injection - attacker-supplied input is interpreted as part of a database query, letting the attacker read or alter data the application never intended to expose. **Command injection** - attacker-supplied input reaches an operating-system shell, letting the attacker run arbitrary commands with the web server's privileges. **Cross-site scripting (XSS)** - attacker-supplied script is executed in another user's browser in the application's origin, giving the script access to that user's session. **CSRF token** - a per-request secret that proves a state-changing request was generated by the application itself, not by a third-party page. **HttpOnly / Secure / SameSite** - cookie flags that block script access, require HTTPS transport, and restrict cross-site sending respectively. **Unsalted MD5** - a password hash computed without a per-user random value, so identical passwords produce identical hashes and rainbow tables crack them instantly. **CVSS** - the Common Vulnerability Scoring System; this report uses version 3.1 base scores with a business-impact adjustment where stated.